

JomCode Coding Bootcamp

Exceptions, logging and debugging

Understand, predict, type, run, then practice.

What you will be able to do

- 01 Catch expected errors narrowly and raise meaningful domain failures.
- 02 Use logging without leaking passwords or connection strings.
- 03 Debug by reproducing, inspecting and testing one cause at a time.

Plain meaning → technical view → use

PLAIN MEANING

Handle a problem without hiding what went wrong.

TECHNICAL VIEW

Exceptions signal expected failures; logging records diagnostic context.

WHEN YOU USE IT

Report an invalid request while keeping credentials out of logs.

TIP

Read the last line of the traceback first, then inspect the failing line.

Worked example 1: Conversion recovery



chapter_11_exceptions_logging_and_debugging.py

```
1 text = "twenty"
2 try:
3     capacity = int(text)
4 except ValueError:
5     print("Enter a whole number")
6 else:
7     print(capacity)
```

PREDICT Which exception should this code catch?

Example 1: output and reason

TERMINAL >_ EXPECTED OUTPUT

Enter a whole number

WHY THIS HAPPENS

A specific ValueError communicates the known conversion boundary without hiding unrelated bugs.

Exercise 1: Integer parser

BUILD

Write `parse_capacity(text)` converting text and enforcing 1–500. Raise `ValueError` for invalid values.

CHECK

"1"/"500" accepted. "0"/"501"/"abc" rejected.

IF STUCK

`int` already raises `ValueError` for invalid numeric text.

Worked example 2: Domain validation



chapter_11_exceptions_logging_and_debugging.py

```
1 def valid_capacity(value):
2     if type(value) is not int or not 1 ≤ value ≤ 500:
3         raise ValueError("Capacity must be an integer from 1 to 500")
4     return value
5
6 print(valid_capacity(20))
```

PREDICT Where should validation happen?

Example 2: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
20. Calling with True or 0 raises ValueError.
```

WHY THIS HAPPENS

Validate near the input boundary, then preserve the same rule in the domain and persistence layers.

Exercise 2: Failure-path check

BUILD

Use try/except to verify that invalid numeric text fails. Do not silently return zero.

CHECK

"abc" raises ValueError. A valid count stays distinguishable from an invalid input.

IF STUCK

An error is a separate outcome, not a fake valid value.

Worked example 3: Safe event logging



chapter_11_exceptions_logging_and_debugging.py

```
1 import logging
2
3 logger = logging.getLogger("workshop_hub")
4 logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
5 workshop_id = "w1"
6 logger.info("Workshop update rejected: id=%s reason=capacity", workshop_id)
```

PREDICT What data must stay out of this log?

Example 3: output and reason

TERMINAL >_ EXPECTED OUTPUT

```
INFO: Workshop update rejected: id=w1 reason=capacity
```

WHY THIS HAPPENS

Passwords, bearer tokens, raw database credentials and full submitted personal records should stay out.

Exercise 3: Debugging record

BUILD

Reproduce an off-by-one loop bug and record the condition, breakpoint state and fix.

CHECK

`range(1,3)` produces only two attempts. `range(1,4)` produces three.

IF STUCK

Write the expected behavior before editing.

Checkpoint

PROVE IT

Invalid input produces a clear error and the debug process leaves evidence.